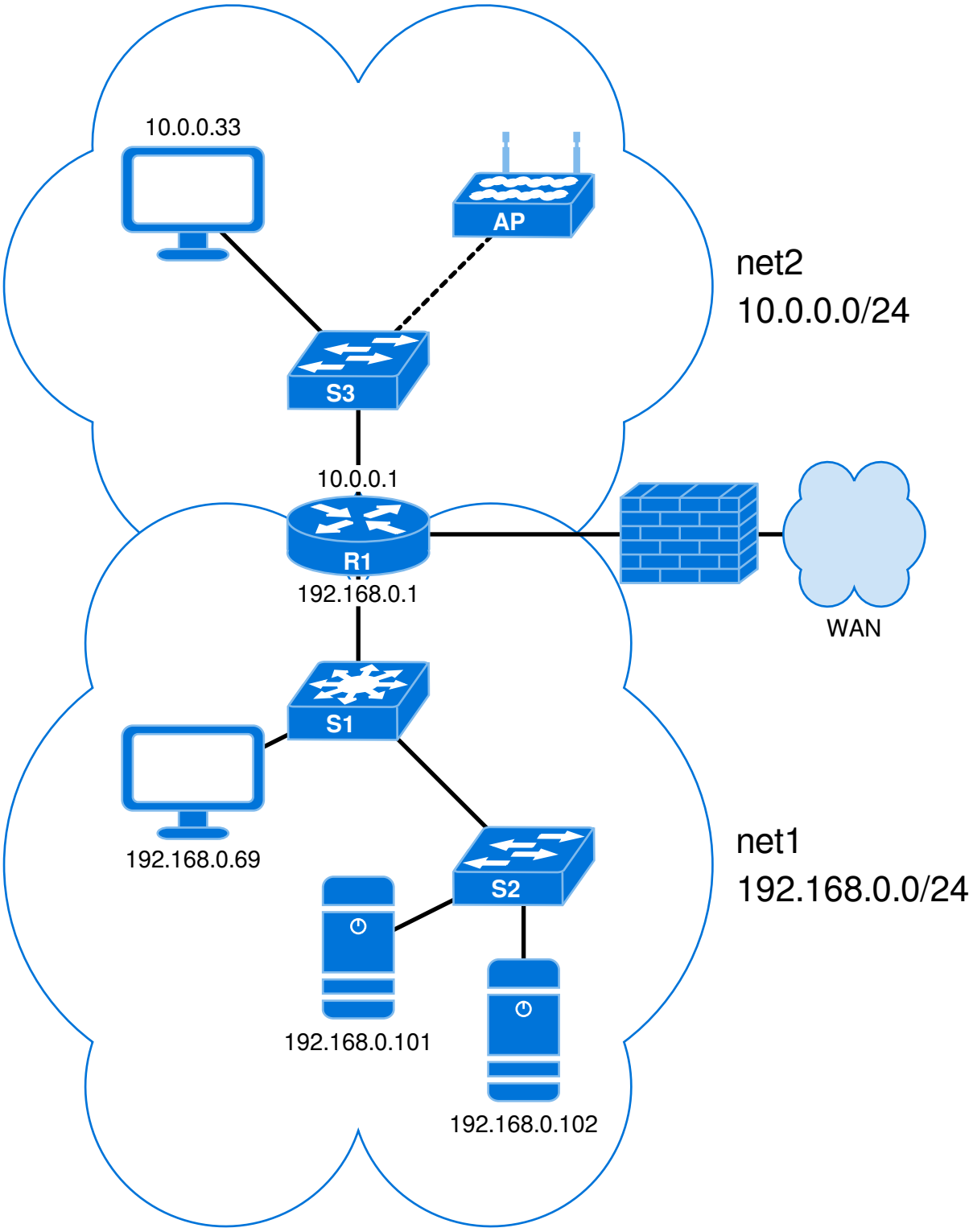
# cntopo 0.0.1

A modular set of CeTZ-powered computer network topology icons with Fletcher integration.



# Contents

Overview . . . . . 3
API . . . . . 3
Main . . . . . 3
Node shapes . . . . . 5
Node classes . . . . . 7
Node details . . . . . 11
Nodes . . . . . 12
Clients . . . . . 14
Miscellaneous . . . . . 16
Shapes . . . . . 17

# Overview

## API

Whenever `..pos` is used, the positional arguments can be one of these:

- none ⇒ defaults to the position `(0,0)` and a preconfigured default size for the specific shape
- `(x, y)` ⇒ sets the position of the icon to the x and y value
- `(x, y), (w, h)` ⇒ sets the position of the icon to the x and y value and its radius to w (width) and h (height)
- `(x, y), r` ⇒ sets the radius of the icon to `(r, r)`

## Main

- fletcher-shapes()
- icons()
- to-fletcher-shapes()

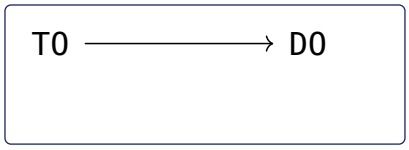
### Variables

- default-nodes
- icon-presets

fletcher-shapes(..args: `TODO`) -> `dict`

Shapes to use in Fletcher diagrams.

```
#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node
#diagram(node((0,0), "TO"), edge("->"), node((2,0), "DO"))
```




**Parameters:**

`..args` (`TODO`) – The same args as in icons

icons(
  stroke,
  fill,
  stroke-inner,
  fill-inner,
  flat
) -> `dict`

Applies styling to all of the CeTZ icons

```
TODO
```

TODO

to-fletcher-shapes(i: `TODO`) -> `dict`

Helper function to convert the CeTZ icons to Fletcher shapes

| | |
|---|---|
| TODO | TODO |

**Parameters:**

`i` (`TODO`) – CeTZ icons

### `default-nodes` `dict`

Default node icons

| | |
|---|---|
| TODO | TODO |

### `icon-presets` `dict`

Icon presets

| | |
|---|---|
| TODO | TODO |

NOTE: All of these shapes can be accessed in these three ways:

```
// Example: rect
node(shape: node-shape-rect)
node(shape: node-shapes.rect)
node(shape: "rect")
```

NOTE: A custom node shape can be passed to the `shape` parameter. It must contain the following signature:

```
#let my-custom-shape = (
  /// width
  sx,
  /// height
  sy,
  /// corner radius
  radius,
  /// stroke
  stroke,
  /// fill
  fill,
  /// if shape is 3d
  flat
) => {
  if not flat {
    cetz.draw.circle(
      (0, 0, 1),
```

```
      radius: (sx, sy),
      stroke: stroke,
      fill: fill,
    )
  }
  cetz.draw.circle(
    (0, 0),
    radius: (sx, sy),
    stroke: stroke,
    fill: fill,
  )
}
```

## Node shapes

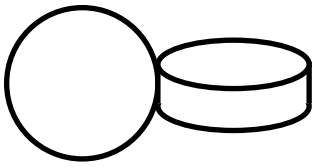
### Variables

- node-shape-circle
- node-shape-hex
- node-shape-square
- node-shape-rect
- node-shape-bridge
- node-shape-firewall
- node-shapes

```
node-shape-circle
```

Circular node shape

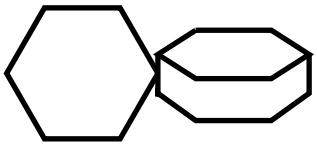
```
#cetz.canvas({
  node((0,0), shape: "circle")
  node((2,0), shape: "circle", flat: false)
})
```



```
node-shape-hex
```

Hexagonal node shape

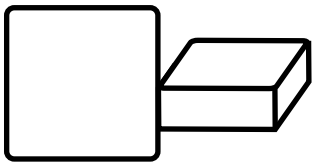
```
#cetz.canvas({
  node((0,0), shape: "hex")
  node((2,0), shape: "hex", flat: false)
})
```



```
node-shape-square
```

Square node shape

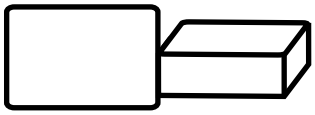
```
#cetz.canvas({
  node((0,0), shape: "square")
  node((2,0), shape: "square", flat: false)
})
```



`node-shape-rect`

Rectangular node shape

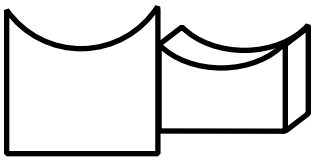
```
#cetz.canvas({
  node((0,0), shape: "rect")
  node((2,0), shape: "rect", flat: false)
})
```



`node-shape-bridge`

Bridge node shape

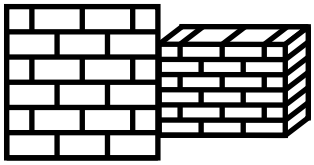
```
#cetz.canvas({
  node((0,0), shape: "bridge")
  node((2,0), shape: "bridge", flat: false)
})
```



`node-shape-firewall`

Firewall node shape

```
#cetz.canvas({
  node((0,0), shape: "firewall")
  node((2,0), shape: "firewall", flat: false)
})
```



`node-shapes` `dict`

Dict containing all of the node shape presets

NOTE: All of these classes can be accessed in these three ways:

```
// Example: router
node(class: node-class-router)
node(class: node-classes.router)
node(class: "router")
```

NOTE: A custom node class can be passed to the `class` parameter. It must contain the following signature:

```
#let my-custom-class = (
  /// parent node width
  sx,
  /// parent node height
  sy,
  /// stroke
  stroke,
  /// fill
  fill
) => {
  cetz.draw.circle(
    (0, 0, 0),
    radius: (sx, sy),
    stroke: stroke,
    fill: fill,
  )
}
```

NOTE: Classes can also be regular content:

```
node(class: [ABC])
```

NOTE: Classes can also be used as details and vice-versa.

NOTE: Classes influence node shape if it's set to `auto`

## Node classes

### Variables

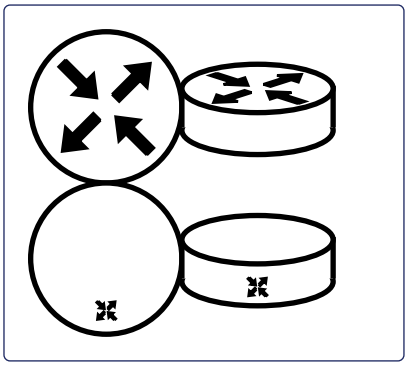
- node-class-router
- node-class-switch
- node-class-hub
- node-class-fe-hub
- node-class-l3-switch
- node-class-ap
- node-class-dual-ap
- node-class-mesh-ap
- node-classes

```
node-class-router
```

Router node class

```
#cetz.canvas({
  node((0,2), class: "router")
  node((2,2), class: "router", flat: false)
```

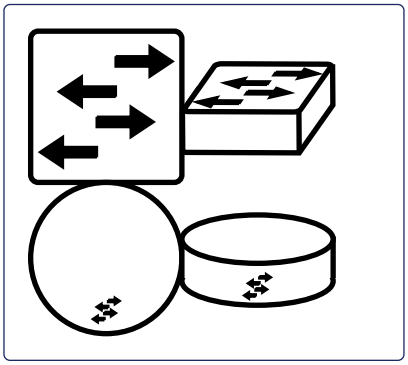
```
  node((0,0), detail: "router")
  node((2,0), detail: "router", flat: false)
})
```



## node-class-switch

Switch node class

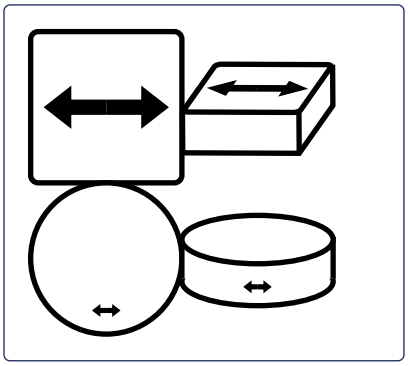
```
#cetz.canvas({
  node((0,2), class: "switch")
  node((2,2), class: "switch", flat: false)
  node((0,0), detail: "switch")
  node((2,0), detail: "switch", flat: false)
})
```



## node-class-hub

Hub node class

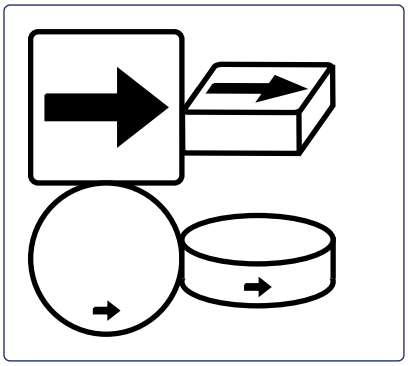
```
#cetz.canvas({
  node((0,2), class: "hub")
  node((2,2), class: "hub", flat: false)
  node((0,0), detail: "hub")
  node((2,0), detail: "hub", flat: false)
})
```



## node-class-fe-hub

Fast ethernet hub node class

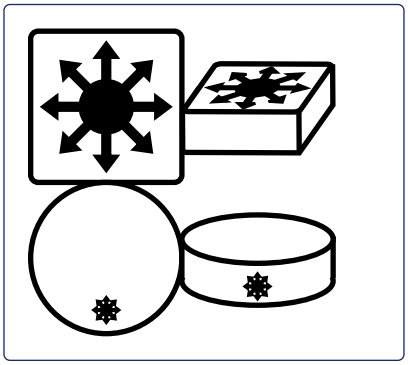
```
#cetz.canvas({
  node((0,2), class: "fe-hub")
  node((2,2), class: "fe-hub", flat: false)
  node((0,0), detail: "fe-hub")
  node((2,0), detail: "fe-hub", flat: false)
})
```



## node-class-l3-switch

Layer 3 switch node class

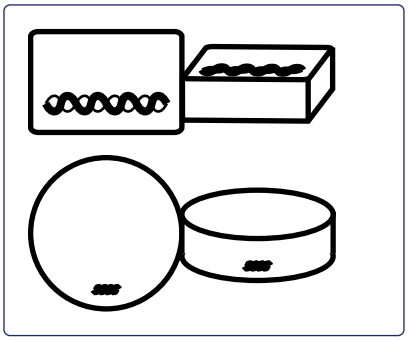
```
#cetz.canvas({
  node((0,2), class: "l3-switch")
  node((2,2), class: "l3-switch", flat: false)
  node((0,0), detail: "l3-switch")
  node((2,0), detail: "l3-switch", flat: false)
})
```



## node-class-ap

Access point node class

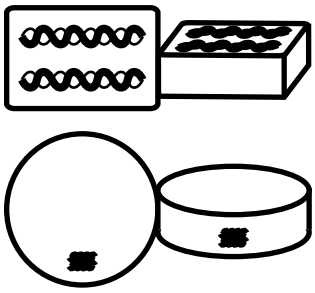
```
#cetz.canvas({
  node((0,2), class: "ap")
  node((2,2), class: "ap", flat: false)
  node((0,0), detail: "ap")
  node((2,0), detail: "ap", flat: false)
})
```



## node-class-dual-ap

Dual access point node class

```
#cetz.canvas({
  node((0,2), class: "dual-ap")
  node((2,2), class: "dual-ap", flat: false)
  node((0,0), detail: "dual-ap")
  node((2,0), detail: "dual-ap", flat: false)
})
```

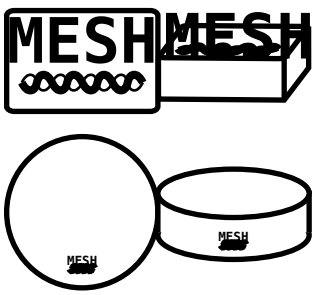


## `node-class-mesh-ap`

Mesh access point node class

As you can see, the text transformation doesn't work yet in 3d

```
#cetz.canvas({
  node((0,2), class: "mesh-ap")
  node((2,2), class: "mesh-ap", flat: false)
  node((0,0), detail: "mesh-ap")
  node((2,0), detail: "mesh-ap", flat: false)
})
```




## `node-classes` `dict`

Dict containing all of the node class presets

NOTE: All of these details can be accessed in these three ways:

```
// Example: secure
node(detail: node-detail-secure)
node(detail: node-details.secure)
node(detail: "secure")
```

NOTE: A custom node detail can be passed to the `detail` parameter. It must contain the following signature:

```
#let my-custom-detail = (
  /// parent node width
  sx,
  /// parent node height
  sy,
  /// stroke
  stroke,
  /// fill
  fill
```

```
) => {
  cetz.draw.circle(
    (0, 0, 0),
    radius: (sx, sy),
    stroke: stroke,
    fill: fill,
  )
}
```

NOTE: Details can also be regular content:

```
node(detail: [ABC])
```

NOTE: Details can also be used as classes and vice-versa.

## Node details

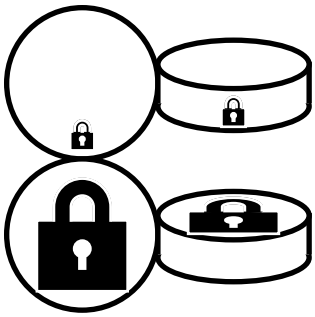
### Variables

- node-detail-secure
- node-detail-cloud
- node-details

```
node-detail-secure
```

Secure node detail

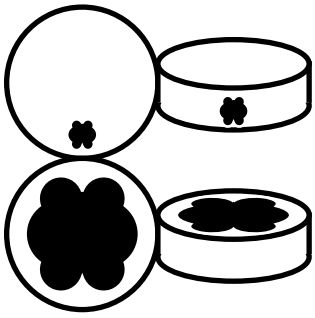
```
#cetz.canvas({
  node((0,2), detail: "secure")
  node((2,2), detail: "secure", flat: false)
  node((0,0), class: "secure")
  node((2,0), class: "secure", flat: false)
})
```



```
node-detail-cloud
```

Cloud node detail

```
#cetz.canvas({
  node((0,2), detail: "cloud")
  node((2,2), detail: "cloud", flat: false)
  node((0,0), class: "cloud")
  node((2,0), class: "cloud", flat: false)
})
```



`node-details` `dict`

Dict containing all of the node detail presets

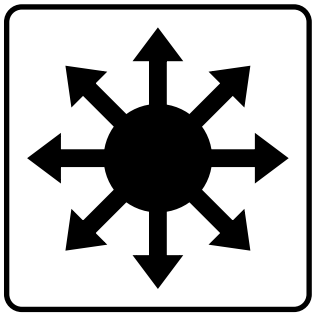
## Nodes

- node()

```
node(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment,
  detail: "secure" "cloud" content none,
  class: "router" "switch" "hub" "fe-hub" "l3-switch" "ap" "dual-ap" "mesh-ap" content none,
  shape: "circle" "hex" "square" "rect" "bridge" "firewall" auto,
  radius: int float relative auto,
  wireless: bool
)
```

Node

```
#cetz.canvas({
  node((0,0), (2,2), class: "l3-switch")
})
```



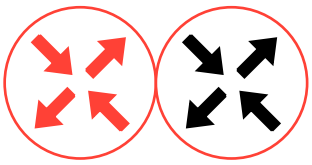
**Parameters:**

`..pos` ( `(x, y)` or `(x, y), (w, h)` ) – The position (and size)

`stroke` ( `stroke` = `stroke-def` ) – Icon outer stroke

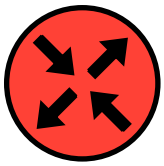
If `node.stroke-inner` and/or `node.fill-inner` are set to `auto`, then this will apply to them as well.

```
#cetz.canvas({
  node((0,0), class: "router", stroke: red)
  node((2,0), class: "router", stroke: red, stroke-inner:
black, fill-inner: black)
})
```



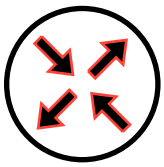
`fill` (`paint` = `fill-def`) – Icon main fill

```
#cetz.canvas({
  node(fill: red, class: "router")
})
```



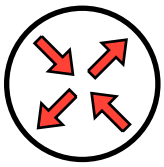
`stroke-inner` (`stroke` or `auto` = `auto`) – Icon inner stroke

```
#cetz.canvas({
  node(stroke-inner: red, class: "router")
})
```



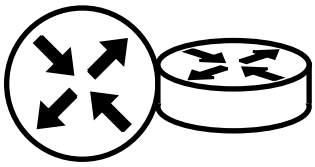
`fill-inner` (`paint` or `auto` = `auto`) – Icon inner fill

```
#cetz.canvas({
  node(fill-inner: red, class: "router")
})
```



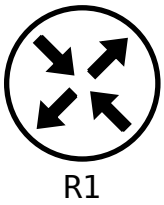
`flat` (`bool` = `true`) – If the icon should be flat or 3d

```
#cetz.canvas({
  node((0,0), flat: true, class: "router")
  node((2,0), flat: false, class: "router")
})
```



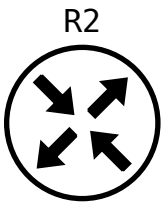
`label` (`str` or `content` or `none` = `none`) – Label

```
#cetz.canvas({
  node(label: "R1", class: "router")
})
```



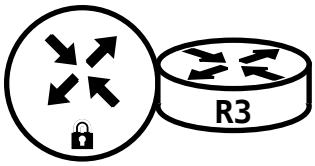

`label-pos` (`alignment` = `bottom`) – Label position

```
#cetz.canvas({
  node(label: "R2", label-pos: top, class: "router")
})
```



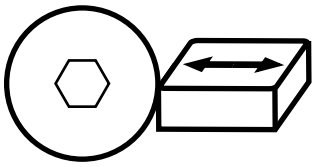
`detail` (`"secure"` or `"cloud"` or `content` or `none` = `none`) – Secondary node icon / text

```
#cetz.canvas({
  node((0,0), detail: "secure", class: "router")
  node((2,0), detail: [R3], flat: false, class: "router")
})
```



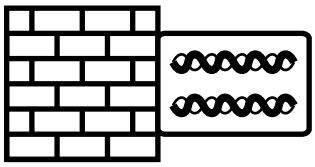

`class` (`"router"` or `"switch"` or `"hub"` or `"fe-hub"` or `"l3-switch"` or `"ap"` or `"dual-ap"` or `"mesh-ap"` or `content` or `none` = `none`) – Main node icon

```
#cetz.canvas({
  node(class: polygon.regular(size: 20pt, vertices: 6))
  node((2,0), class: "hub", flat: false)
})
```



`shape` (`"circle"` or `"hex"` or `"square"` or `"rect"` or `"bridge"` or `"firewall"` or `auto` = `auto`) – Node shape

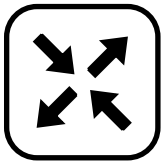
```
#cetz.canvas({
  node(shape: "firewall")
  node((2, 0), class: "dual-ap", shape: auto)
})
```



`radius` (`int` or `float` or `relative` or `auto` = `auto`) – Node radius.

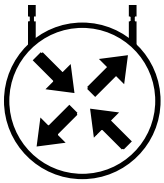
Only works on the shapes `square` or `rect`

```
#cetz.canvas({
  node(shape: "square", radius: 20%, class: "router")
})
```



`wireless` (`bool` = `false`) – If the wireless antennas should be added

```
#cetz.canvas({
  node(wireless: true, class: "router")
})
```



## Clients

- monitor()
- server()

```
monitor(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment
)
```

Monitor

```
#cetz.canvas({
  monitor(label: "My PC")
})
```

**Parameters:**

`..pos` ( `(x, y)` or `(x, y), (w, h)` ) – The position (and size)

`stroke` ( `stroke` = `stroke-def`) – Icon outer stroke

`fill` ( `paint` = `fill-def`) – Icon main fill

`stroke-inner` ( `stroke` or `auto` = `auto`) – Icon inner stroke

`fill-inner` ( `paint` or `auto` = `auto`) – Icon inner fill

`flat` ( `bool` = `true`) – Not implemented yet

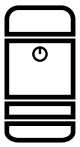
`label` ( `str` or `content` or `none` = `none`) – Label

`label-pos` ( `alignment` = `bottom`) – Label position

```
server(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment
)
```

Server

```
#cetz.canvas({
  server(label: "Some greek god's name")
})
```



Some greek god's name

**Parameters:**

`..pos` ( `(x, y)` or `(x, y), (w, h)` ) – The position (and size)

`stroke` ( `stroke` = `stroke-def` ) – Icon outer stroke

`fill` ( `paint` = `fill-def` ) – Icon main fill

`stroke-inner` ( `stroke` or `auto` = `auto` ) – Icon inner stroke

`fill-inner` ( `paint` or `auto` = `auto` ) – Icon inner fill

`flat` ( `bool` = `true` ) – Not implemented yet

`label` ( `str` or `content` or `none` = `none` ) – Label

`label-pos` ( `alignment` = `bottom` ) – Label position

## Miscellaneous

- lock()

### Variables

- cloud

```
lock(
  ..pos: (x, y) (x, y), (w, h),
  stroke: stroke,
  fill: paint,
  stroke-inner: stroke auto,
  fill-inner: paint auto,
  flat: bool,
  label: str content none,
  label-pos: alignment
)
```

Lock

```
#cetz.canvas({
  lock(label: "Don't even try")
})
```

Don't even try

**Parameters:**

`..pos` ( `(x, y)` or `(x, y), (w, h)` ) – The position (and size)

`stroke` ( `stroke` = `stroke-def` ) – Icon outer stroke

`fill` (`paint` = `fill-def`) – Icon main fill

`stroke-inner` (`stroke` or `auto` = `auto`) – Icon inner stroke

`fill-inner` (`paint` or `auto` = `auto`) – Icon inner fill

`flat` (`bool` = `true`) – Not implemented yet

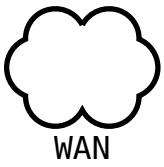
`label` (`str` or `content` or `none` = `none`) – Label

`label-pos` (`alignment` = `bottom`) – Label position

```
cloud
```

Cloud

```
#cetz.canvas({
  cloud(label: "WAN")
})
```




## Shapes

- antennas()
- arrow()
- arrows()
- wireless-wave()

antennas(..pos: `(x, y)` `(x, y), (w, h)`, spacing: `int` `length` `auto`, stroke: `stroke`, fill: `paint` `auto`)

Antennas shape

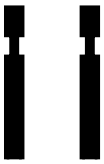
```
#cetz.canvas({
  antennas((0,0), (3,3))
})
```

**Parameters:**

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`spacing` (`int` or `length` or `auto` = `auto`) – Spacing between the antennas

```
#cetz.canvas({
  antennas((0,0), (3,3), spacing: 1)
})
```



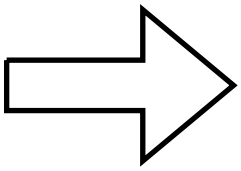
`stroke` (`stroke` = `stroke-def`) – Antennas stroke

`fill` (`paint` or `auto` = `auto`) – Antennas fill

```
arrow(
  ..pos: (x, y) (x, y), (w, h),
  dir: direction,
  connect: bool,
  ratio-len: float,
  ratio-width: float,
  stroke: stroke,
  fill: paint
)
```

Arrow shape

```
#cetz.canvas({
  arrow((0,0), (3,2))
})
```



**Parameters:**

`..pos` (`(x, y)` or `(x, y), (w, h)`) – The position (and size)

`dir` (`direction` = `ltr`) – Direction the arrow points to

```
#cetz.canvas({
  arrow(dir: ttb)
})
```

`connect` (`bool` = `true`) – Whether to connect the tail of the arrow

```
#cetz.canvas({
  arrow(connect: false)
})
```

`ratio-len` (`float` = `3 / 5`) – Tail length to head length ratio

```
#cetz.canvas({
  arrow(ratio-len: 1 / 3)
})
```

`ratio-width` (`float` = `1 / 3`) – Tail width to head width ratio

```
#cetz.canvas({
  arrow(ratio-width: 2 / 3)
})
```

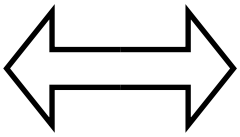
`stroke` (`stroke` = `stroke-def`) – Arrow stroke

`fill` ( `paint` = `fill-def` ) – Arrow fill

```
arrows(
  ..pos: (x, y) (x, y), (w, h),
  type: "cross" "two" "two-v",
  ratio-len: float,
  ratio-width: float,
  stroke: stroke,
  fill: paint
)
```

Arrows shape

```
#cetz.canvas({
  arrows((0,0), (3,3))
})
```

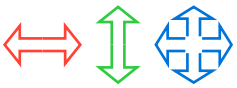


**Parameters:**

`..pos` ( `(x, y)` or `(x, y), (w, h)` ) – The position (and size)

`type` ( `"cross"` or `"two"` or `"two-v"` = `"two"` ) – How arrows should be composed

```
#cetz.canvas({
  arrows((0,0), type: "two", stroke: red)
  arrows((1,0), type: "two-v", stroke: green)
  arrows((2,0), type: "cross", stroke: blue)
})
```



`ratio-len` ( `float` = `3 / 5` ) – Tail length to head length ratio

```
#cetz.canvas({
  arrows(ratio-len: 1 / 3)
})
```

`ratio-width` ( `float` = `1 / 3` ) – Tail width to head width ratio

```
#cetz.canvas({
  arrows(ratio-width: 2 / 3)
})
```

`stroke` ( `stroke` = `stroke-def` ) – Arrow stroke

`fill` ( `paint` = `fill-def` ) – Arrow fill

wireless-wave(..pos: `(x, y)` `(x, y), (w, h)`, stroke: `stroke`, stroke-inner: `stroke`, n: `int`)

Wireless wave shape

```
#cetz.canvas({
  wireless-wave((0,0), (2,0.2))
})
```

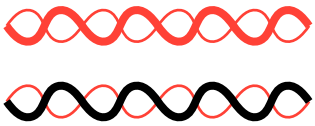
**Parameters:**

`..pos` ( `(x, y)` or `(x, y), (w, h)` ) – The position (and size)

`stroke` ( `stroke` = `stroke-def` ) – Wave stroke

If `inner-stroke` is set to auto, the same paint gets applied

```
#cetz.canvas({
  wireless-wave((0,1), (2,0.2), stroke: red)
  wireless-wave((0,0), (2,0.2), stroke: red, stroke-
inner: black + 3pt)
})
```



`stroke-inner` ( `stroke` = `auto` ) – Inner wave stroke

```
#cetz.canvas({
  wireless-wave((0,0), (2,0.2), stroke-inner: blue + 5pt)
})
```

`n` ( `int` = `4` ) – Number of oscillations

```
#cetz.canvas({
  wireless-wave((0,0), (3,0.2), n: 12)
})
```